Can an MLP Learn Complex Arithmetic? A Neural Calculator for $+$, $-$, $\times$ and $\div$ on Complex Numbers

Nithin D H
Indian Institute of Technology Kanpur (IIT-Kanpur)
Kanpur, India

Abstract

Neural networks are universal approximators, but approximating a function is not the same as computing it exactly. We test this on a small, fully controlled problem: a multilayer perceptron (MLP) that takes two complex numbers and an operation, and outputs the real and imaginary parts of the result.

*We train two models on synthetic data. A 4.7k-parameter baseline uses a scalar operation code and unscaled targets. A 103k-parameter improved model uses one-hot operation encoding, standardised inputs and outputs, batch normalisation, Huber loss and 5× more data. On ten test expressions the improved model cuts the mean relative error from 7.3% to **2.3%** for addition, 12.2% to **2.6%** for subtraction, 76.9% to **17.7%** for multiplication and 84.8% to **32.2%** for division. The ranking follows the mathematics: linear operations are learned almost exactly, bilinear products only approximately, and the rational division is the hardest.*

1. Introduction

Motivation. The universal approximation theorem [2] guarantees that an MLP *can* represent any continuous function on a bounded domain, but says nothing about how easily it learns one. Complex arithmetic is an ideal probe, because ground truth is free and exact and the four operations increase steadily in difficulty:

- $a \pm b$ is *linear* in the inputs;
- $a \times b = (a_r b_r - a_i b_i) + i(a_r b_i + a_i b_r)$ is *bilinear*;
- $a \div b = \dfrac{(a_r b_r + a_i b_i) + i(a_i b_r - a_r b_i)}{b_r^2 + b_i^2}$ is *rational*, with steep gradients near $|b| \to 0$.

What we do. We frame the calculator as regression from $(a_r, a_i, b_r, b_i, \text{op})$ to (r_r, r_i), train a baseline and an improved MLP, and analyse their errors by operation.

2. Related Work

Trask *et al.* [6] showed that standard MLPs extrapolate arithmetic poorly and proposed the Neural Arithmetic Logic Unit, which builds addition and multiplication (the latter in log space) into the architecture. Complex-valued networks [5] use complex weights and activations instead. We deliberately use a plain real-valued MLP to measure how far careful data design and normalisation can go without architectural priors.

Table 1. Configurations. Parameter counts verified by instantiating the models.

	Baseline	Improved
Input	4 values + scalar op code	4 values + one-hot op
Scaling	inputs min-max to $[-1, 1]$	inputs and outputs z-scored
Network	5→64→64→2	8→256→256→128→2, BN
Params	4,674	102,530
Data	100k samples	500k samples
Loss	MSE	Huber ($\delta = 1$)
Optimiser	Adam 10^{-3}, plateau decay	Adam 3×10^{-3}, cosine [4]
Epochs	60, batch 512	150, batch 1024, clip 1.0

3. Method

3.1. Synthetic data

Each of a_r, a_i, b_r, b_i is drawn uniformly from $[-10, 10]$, and the operation uniformly from $\{+, -, \times, \div\}$. Targets come from the closed-form expressions above. For division, near-zero denominators are re-sampled (baseline: $|b_r|, |b_i| < 0.1$; improved: $|b| < 0.5$). We use a 90/10 train/validation split.

3.2. Models

Figure 1 and Table 1 describe the two models.

Why these changes. (i) A scalar operation code (0.25, 0.5, 0.75, 1.0) implies a false ordering, as if $\div$ were "four times" $+$; one-hot encoding removes it. (ii) Products reach ± 200 while sums stay within ± 20, so unscaled targets let $\times$ and $\div$ dominate the MSE; standardising the outputs balances the operations. (iii) Huber loss [3] and gradient clipping limit the influence of large division errors. (iv) More width, depth and data give the network the capacity to approximate the bilinear terms. Both models use ReLU and Kaiming

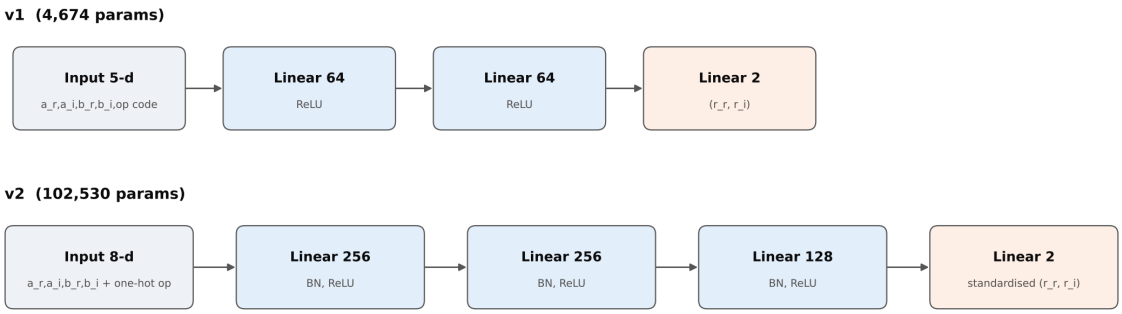
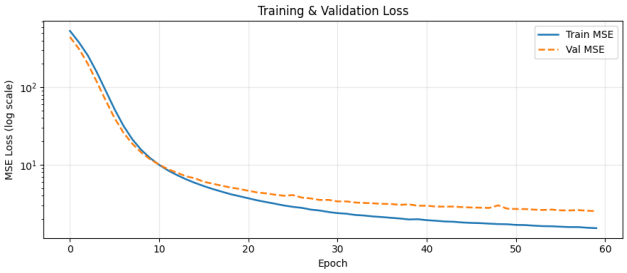


Figure 1. The two MLPs. The improved model adds width, depth, batch normalisation, one-hot operation encoding and output standardisation.



(a) Baseline (MSE)



(b) Improved (Huber, standardised targets)

Figure 2. Training and validation loss (log scale).

initialisation [1], and the improved model restores its best validation checkpoint.

4. Results and Discussion

4.1. Training

Both models converge smoothly (Fig. 2). The two curves use different losses on differently scaled targets, so their values are not directly comparable.

4.2. Ten test expressions

Table 2 lists predictions on ten hand-picked expressions covering all operations and several edge cases (pure imaginary numbers, a conjugate pair, $i \times i$). Relative error is $|\hat{r} - r|/|r|$.

4.3. Analysis

Linear operations are easy. A ReLU MLP can represent $a \pm b$ almost exactly, and the improved model gets both to about 2%.

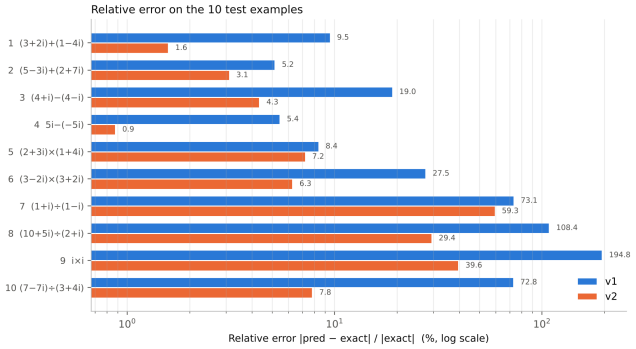


Figure 3. Relative error per example (log scale). The improved model is better on all ten.

Products need cross terms. Multiplication requires $a_r b_r$ and $a_i b_i$, which a piecewise-linear network can only approximate. Cancellation cases (the conjugate product, $i \times i$) are worst in *relative* terms, because the true result has a zero component: a small absolute error becomes a large relative one.

Division is hardest. The $1/|b|^2$ factor is steep for small $|b|$. The baseline essentially fails on division (73–108%). The improved model reduces this to 8–59%, but example 7, where $|b| = \sqrt{2}$ is small, is still poor.

What drove the improvement? Encoding, normalisation, capacity (22× more parameters) and data (5×) were changed together, so their individual contributions are not separated here.

Reproducibility. Re-running the seeded baseline on a CPU reproduced the reported predictions to within about 1% (e.g.example 9: 195.47% vs. 194.85%), the difference coming from CPU versus GPU floating-point behaviour.

5. Conclusion & Future Work

An MLP learns complex addition and subtraction to within a few percent, but multiplication and especially division remain approximate even after careful encoding and normalisation. It is a useful approximator, not an exact calculator.

Next steps. (i) Evaluate on thousands of random held-out

Table 2. Predictions of both models on the same ten inputs.

#	Expression	Exact	Baseline prediction	Rel. err.	Improved prediction	Rel. err.
1	$(3+2i)+(1-4i)$	$4-2i$	$3.870-2.405i$	9.51%	$4.037-2.060i$	1.58%
2	$(5-3i)+(2+7i)$	$7+4i$	$7.045+4.413i$	5.16%	$6.792+3.861i$	3.11%
3	$(4+i)-(4-i)$	$2i$	$-0.158+2.346i$	18.99%	$-0.086+1.991i$	4.34%
4	$5i-(-5i)$	$10i$	$0.059+9.458i$	5.45%	$-0.086+10.019i$	0.88%
5	$(2+3i)\times(1+4i)$	$-10+11i$	$-9.487+9.865i$	8.38%	$-10.535+10.069i$	7.22%
6	$(3-2i)\times(3+2i)$	13	$13.415-3.551i$	27.50%	$13.170+0.797i$	6.27%
7	$(1+i)\div(1-i)$	i	$-0.347+0.356i$	73.15%	$-0.378+1.457i$	59.30%
8	$(10+5i)\div(2+i)$	5	$-0.421+0.090i$	108.43%	$6.352+0.581i$	29.42%
9	$i\times i$	-1	$0.904-0.413i$	194.85%	$-0.770-0.322i$	39.59%
10	$(7-7i)\div(3+4i)$	$-0.28-1.96i$	$-0.389-0.523i$	72.81%	$-0.415-2.035i$	7.81%

Table 3. Mean relative error by operation (from Table 2).

Operation (examples)	Baseline	Improved
Addition (1, 2)	7.3%	**2.3%**
Subtraction (3, 4)	12.2%	**2.6%**
Multiplication (5, 6, 9)	76.9%	**17.7%**
Division (7, 8, 10)	84.8%	**32.2%**

samples per operation, since ten examples give high-variance averages; (ii) represent numbers in polar or log form so that $\times$ and $\div$ become additions; (iii) add multiplicative or gated units, as in NALU [6]; and (iv) run an ablation over the four improvements.

Acknowledgement. This work was carried out as part of EE655 Computer Vision & Deep Learning at IIT Kanpur under the supervision of Dr. Koteswar Rao Jerripothula.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Int. Conf. Comput. Vis.*, 2015.

[2] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[3] P. J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[4] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *Int. Conf. Learn. Represent.*, 2017.

[5] C. Trabelsi et al. Deep complex networks. In *Int. Conf. Learn. Represent.*, 2018.

[6] A. Trask, F. Hill, S. Reed, J. Rae, C. Dyer, and P. Blunsom. Neural arithmetic logic units. In *Adv. Neural Inform. Process. Syst.*, 2018.